CS/CE 224/227: Object Oriented Programming and Design Methodologies

# Week 08/Unit 08: Operator Overloading

Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk
)

# Unit Outline

1. Operator Overloading – Intro
2. Overloading Unary Operators
3. Overloading Binary Operators
4. Assignment and Comparison
5. Overloading the extraction and insertion operators.
6. Summary

# Overloading Operators – Introduction

- In earlier units, you have seen function overloading.
- Overloaded functions are distinguished by the number, type or order of the function parameters – also called the function signature.
- C++ also allows for **operator** overloading.
- What this means is – you can assign a different operation for a common operator such as +, -, *, /, ++, >, <, etc.
- These additional definitions are useful for user-defined data types.

# Operator Overloading

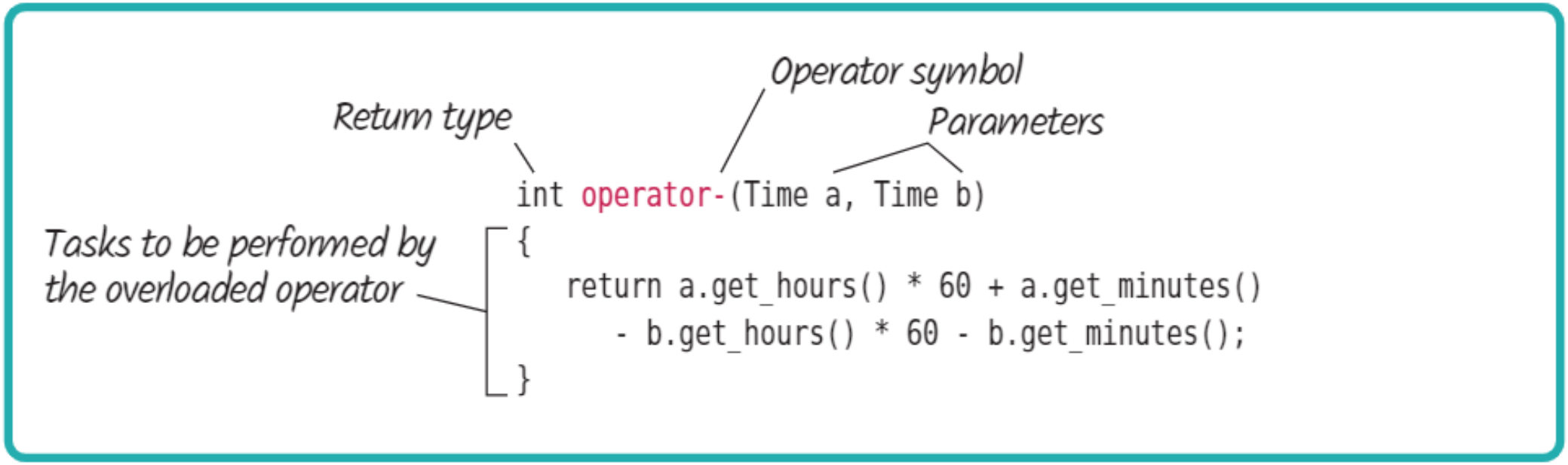
- Consider the `Distance` class defined earlier.

```
class Distance                          //English Distance class
   {
   private:
      int feet;
      float inches;
   public:
      void setdist(int ft, float in)  //set Distance to args
         { feet = ft; inches = in; }
```

- How about adding two distance objects to produce a third distance object?
- In other words, how about `d3 = d1 + d2`? [Think: How will this work?]

# Operator Overloading

- The keyword `operator` is used to overload an operator.
- This is used in a function declaration as follows:



# Operator Overloading – Unary Operators

- Unary Operators (++)
- Consider the class `Counter`:

```
class Counter
{
    private:
        unsigned int count; //count
    public:
    Counter() : count(0) //constructor
    { cout << "Constructor being executed now" <<endl; }
    void inc_count() //increment count
    { count++; }
    int get_count() //return count
    { return count; }
};
```

# Overloading the ++ operator

- Let’s say we want to overload the ‘++’ operator.
- We do that by defining a new function:

```
void operator ++ ()
   {
   ++count;
   }
```

- A call to the operator ++ is as follows:
- `Counter c1; ++c1;`

# Overloading the ++ Operator

- The only way the compiler can distinguish between overloaded functions is by looking at the data types and the number of their arguments.
- In the same way, the only way it can distinguish between overloaded operators is by looking at the data type of their operands.
- If the operand is a basic type such as an int, as in `++intvar`; then the compiler will use its built-in routine to increment an int.
- But if the operand is a Counter variable, the compiler will know to use our user-written `operator++()` instead.

# Overloaded Operator ++ - Figure

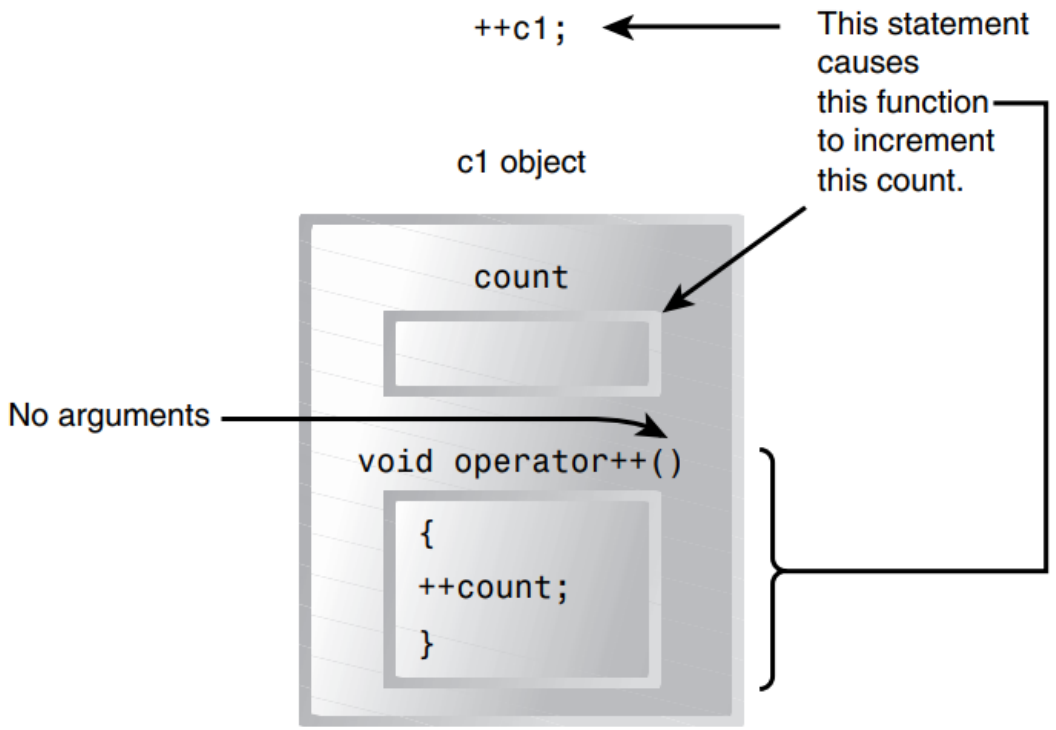



**FIGURE 8.1**

*Overloaded unary operator: no arguments.*

# Operator Overloading – Return Type

- You can also overload an operator to return a type as follows:

```
Counter operator ++ ()    //increment count
   {
   ++count;               //increment count
   Counter temp;          //make a temporary Counter
   temp.count = count;    //give it same value as this obj
   return temp;           //return the copy
   }
```

- Usage:

```
Counter c1, c2; c2 = ++c1;
```

# An Improved ++ function.

```
Counter operator ++ ()      //increment count
   {
   ++count;                    //  increment count, then return
   return Counter(count);      //  an unnamed temporary object
   }                           //  initialized to this count

 class Counter
    {
    private:
       unsigned int count;          //count
    public:
     Counter(int c) : count(c)    //constructor, one arg
        {  }
```

# Overloading postfix (++)

- To overload the postfix ++ operator in C++, you need to distinguish it from the prefix ++ operator.
- The postfix version takes a dummy integer parameter to differentiate it, whereas the prefix version does not take any parameters.
- Here's an example of how you can overload the postfix ++ operator:
  - Define the operator function as a member function.
  - Use the dummy int parameter to signal the postfix version.
  - Return the object in its original state before the increment, typically by returning a copy of the current object.

# Overloading postfix (++)

```
Counter operator ++ (int)    //increment count (postfix)
   {                         //return an unnamed temporary
   return Counter(count++); //object initialized to this
   }                         //count, then increment count


  c2 = c1++;                                  //c1=3, c2=2 (postfix)
```

# Prefix vs Postfix Increment

Now there are two different declarators for overloading the ++ operator. The one we've seen before, for prefix notation, is

```
Counter operator ++ ()
```

The new one, for postfix notation, is

```
Counter operator ++ (int)
```

The only difference is the `int` in the parentheses. This `int` isn't really an argument, and it doesn't mean integer. It's simply a signal to the compiler to create the postfix version of the operator. The designers of C++ are fond of recycling existing operators and keywords to play multiple roles, and `int` is the one they chose to indicate postfix. (Well, can you think of a better syntax?) Here's the output from the program:

# Decrement Operators

- Does the same mechanism applicable to increment operators also work on the decrement (--) operators.
- Write a program to test overloading of the decrement operators.

# Overloading Binary Operators

- Addition

```
Distance operator + ( Distance ) const;  //add 2 distances

Distance Distance::operator + (Distance d2) const  //return sum
   {
   int f = feet + d2.feet;          //add the feet
   float i = inches + d2.inches;    //add the inches
   if(i >= 12.0)                    //if total exceeds 12.0,
      {                             //then decrease inches
      i -= 12.0;                    //by 12.0 and
      f++;                          //increase feet by 1
      }                             //return a temporary Distance
   return Distance(f,i);            //initialized to sum
   }
```

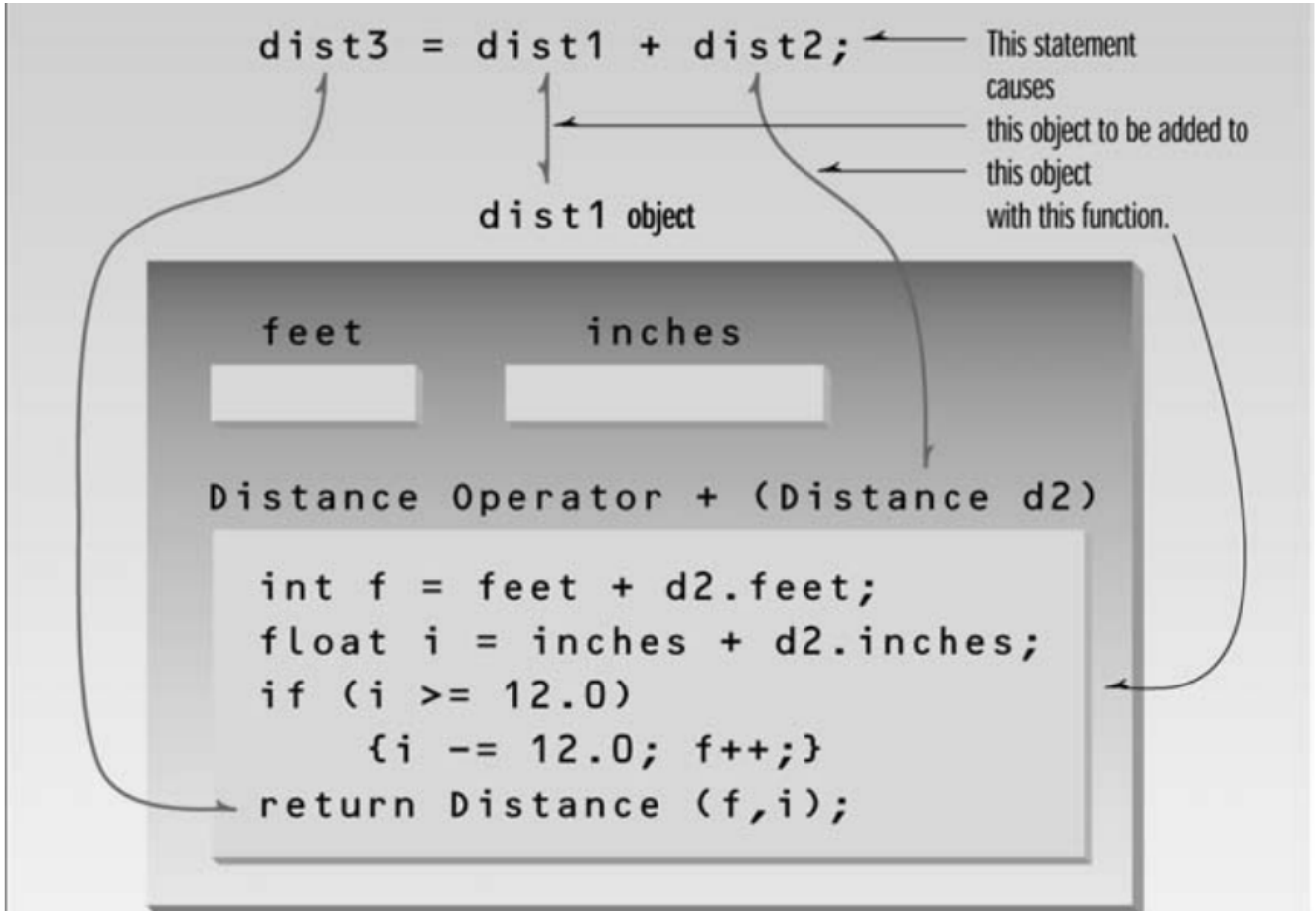
# Main function

```
int main()
   {
   Distance dist1, dist3, dist4;    //define distances
   dist1.getdist();                 //get dist1 from user

   Distance dist2(11, 6.25);        //define, initialize dist2

   dist3 = dist1 + dist2;           //single '+' operator

   dist4 = dist1 + dist2 + dist3;   //multiple '+' operators
                                    //display all lengths
   cout << "dist1 = ";  dist1.showdist(); cout << endl;
   cout << "dist2 = ";  dist2.showdist(); cout << endl;
   cout << "dist3 = ";  dist3.showdist(); cout << endl;
   cout << "dist4 = ";  dist4.showdist(); cout << endl;
   return 0;
   }
```

# Points to Remember

- To add two distances, you can make a function with the following declaration:
- `Distance operator+(Distance d1, Distance d2)`
- This declaration states that the distances d1 and d2 will be added and the resulting distance will be returned as distance d3.
- However, the function can also be declared as:
- `Distance Distance::operator+(Distance d2)`
- Here d1 is the implicit parameter.
- Both types of functions can be called with `d3 = d1 + d2;`

# Figure


dist3 = dist1 + dist2;
This statement causes this object to be added to this object with this function.
dist1 object
feet
inches
Distance Operator + (Distance d2)
int f = feet + d2.feet;
float i = inches + d2.inches;
if (i >= 12.0)
{i -= 12.0; f++;}
return Distance (f,i);

# Some More Points

- In the operator+() function, the left operand is accessed directly—since this is the object of which the operator is a member—using feet and inches.
- The right operand is accessed as the function's argument, as d2.feet and d2.inches.
- We can generalize and say that an overloaded operator always requires one less argument than its number of operands, since one operand is the object of which the operator is a member.
- That's why unary operators require no arguments.

# Overloading Binary Operators

- Similar to the previous slide, any binary operator can be overloaded.
- You may go ahead and overload +, -, *, /, +=, >, <, etc.
- Some key rules:

1. **Almost any operator that exists, can be overloaded.**
2. **You must only overload operators that exists.**
3. **At least one of the arguments in an overloaded operator must be a user defined type.**
4. **It is not possible to change the number of operands that an operator supports.**
5. **The precedence rules (associativity, precedence) between operators cannot be changed.**

# Example: A Time Class (Horstmann)

- Given two Time Objects, we are interested in overloading operators for Time Objects.

```
Time morning(9, 30);
Time lunch(12, 0);
int minutes_to_lunch = lunch - morning;
```

```
class Time
{
public:
   Time();
   Time(int h, int m);
   int get_hours() const;
   int get_minutes() const;
private:
   int hours;
   int minutes;
};
```

# Arithmetic Operators – The `Time` Class

- Subtraction – Returns number of minutes between two time objects

```
int operator-(Time a, Time b)
{
   return a.get_hours() * 60 + a.get_minutes()
      - b.get_hours() * 60 - b.get_minutes();
}
```

- Addition – adding minutes to a `Time` and returning the new `Time`

```
Time operator+(Time a, int minutes)
{
   int result_minutes = a.get_hours() * 60 + a.get_minutes() + minutes;
   return Time((result_minutes / 60) % 24, result_minutes % 60);
}
```

# Comparison Operators – The `Time` Class

```
bool operator==(Time a, Time b)
{
    return a - b == 0;
}
bool operator!=(Time a, Time b)
{
   return a - b != 0;
}

bool operator<(Time a, Time b)
{
   return a - b < 0;
}
```

- The == Operator
- The != Operator
- The < Operator

# Overloading the extraction and insertion operators - << and >> operators

- To overload the extraction and insertion operators, we need to have an idea of what input and output streams are.
- Please read Chapter 8 of Cay Horstmann's book for the same.
- Pdf attached.

# Overloading the extraction and insertion operators - << and >> operators

```
ostream& operator<<(ostream& out, Time a)
{
   out << a.get_hours() << ":"
      << setw(2) << setfill('0')
      << a.get_minutes();
   return out;
}
```

```
cout << noon << "\n";

really means

(cout << noon) << "\n";

that is

operator<<(cout, noon) << "\n"
```

- Here `ostream` represents and output stream
- This function accepts an output stream as a parameter, along with a `Time` object to be sent to the output stream
- It then returns the output stream by reference with the time object printed.
- The `setw` and `setfill` are functions that act as fillers for the ostream.

# Overloading the extraction and insertion operators - << and >> operators

```
istream& operator>>(istream& in, Time& a)
{
   int hours;
   char separator;
   int minutes;
   in >> hours;
   in.get(separator); // Read : character
   in >> minutes;
   a = Time(hours, minutes);
   return in;
}
```

# Summary

- Operator overloading is a useful feature in C++.
- To overload an operator, we use the operator keyword followed by the operator.
- There are certain rules and restrictions applicable to operator overloading.